

Conditions and Patterns for Achieving Convergence in OT-Based Co-Editors

Yi Xu and Chengzheng Sun, *Member, IEEE*

Abstract—In this paper, we report our discovery of general transformation conditions and patterns underlying a range of Operational Transformation (OT) systems for achieving convergence. Based on these conditions and patterns, we have established convergence correctness of seven existing OT systems, and designed two new OT systems with a unique combination of novel features for supporting advanced collaborative applications. These results contribute to the advancement of OT knowledge, technology, and OT-based collaboration editing systems.

Index Terms—Operational transformation, consistency maintenance, collaborative editing systems

1 INTRODUCTION

CONSISTENCY maintenance is a main challenge in building real-time collaborative applications over high latency communication environments like the Internet [5], [17], [21]. Due to its non-blocking, fine-grained concurrency, and unconstrained interaction properties, Operational Transformation (OT) is particularly suitable for consistency maintenance in such environments, and has been increasingly adopted in industrial applications, e.g. Google Docs,¹ IBM OpenCoWeb,² and Codoxword.³

In OT-based real-time collaborative editing applications, multiple users may freely and simultaneously edit any data objects, and editing operations may be transformed and executed in different orders at different collaborating sites [1], [3], [4], [17], [22], [29]. Document *convergence* is a key consistency requirement, i.e., the final documents at all sites must be *identical* after executing the same group of operations [22]. Past research has discovered important transformation properties for achieving convergence in OT systems. In the first OT system [4], a transformation property, *Convergence Property 1 (CP1)*,⁴ was identified, which ensures the same document is produced by executing two concurrent operations in different orders. Follow-up research found CP1 was insufficient to ensure convergence under certain circumstances, and added another transformation property, *Convergence Property 2 (CP2)*,⁵ which ensures the same

operation is produced in transforming one operation against two other operations in different orders (precise definitions of CP1 and CP2 are given in Section 2.3).

An OT system consists of two key components: one is the control algorithms that are generic and responsible for controlling the transformation process; the other is transformation functions that are application-specific and performing real transformations between operations [21], [22]. Control algorithms and transformation functions work together to jointly meet convergence and other correctness requirements. Generally, there exist two approaches to achieving convergence in an OT system: one is to design transformation functions capable of preserving CP1 and/or CP2; the other is to design generic control algorithms capable of avoiding CP1 and/or CP2 [24]. Past research has explored various approaches to preserving or avoiding CP1 and/or CP2, and found that it is relatively easy to preserve CP1 by transformation functions and to avoid CP2 by control algorithms. Multiple CP1-preserving transformation functions for a variety of data and operation models have been designed [2], [12], [15], [24], [29]; and numerous control algorithms capable of avoiding CP2 have been invented [9], [11], [18], [19], [22], [25], [28]. There were some attempts to design functions for preserving CP2 in character-wise text editing systems [8], [13], but avoiding CP2 by control algorithms has been favored due to its advantage of being independent of data and operation models and hence applicable for a wide range of applications. OT systems for advanced collaborative applications, e.g., 2D spreadsheets [26], and 3D digital media design systems [1], [23], have all been designed with a combination of transformation functions for preserving CP1 and control algorithms for avoiding CP2 [25].

Despite the wide adoption of the CP2-avoidance strategy in OT systems, the basic conditions and mechanisms for CP2-avoidance have not been well-understood. Most OT systems that claimed CP2-avoidance had justified their claims by showing the absence of transforming an operation against the same pair of operations in different orders, which is intuitively derived from the CP2 definition [9], [18], [28]. However, more recent research has found the mere absence of transforming the same pair of operations in

1. <http://docs.google.com>

2. <https://github.com/openweb/coweb#readme>

3. <http://www.codoxware.com>

4. CP1 was named as *Transformation Property (TP)* in [4], and *Transformation Property 1 (TP1)* in [15].

5. CP2 was named as *Transformation Property 2 (TP2)* in [15], which is the same as *Transpose Property 5* proposed in [12].

• The authors are with the School of Computer Engineering, Nanyang Technological University, Singapore 639798.
E-mail: {Xuyi, CZSun}@ntu.edu.sg.

Manuscript received 16 Sept. 2014; revised 28 Feb. 2015; accepted 10 Mar. 2015. Date of publication 12 Mar. 2015; date of current version 12 Feb. 2016.

Recommended for acceptance by H. Jin.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TPDS.2015.2412938

different orders is insufficient to avoid CP2 (see [30] for a counter-example), and a correct condition of avoiding CP2 is: to avoid transforming two operations in different contexts [25]. The discovery of this condition has advanced OT knowledge, but also casts doubt on the CP2-avoidance claims of prior OT systems, which made us curious to find out whether those systems are really able to avoid CP2.

Resolving this doubt is also relevant to the correctness of a large body of OT literature covering multiple OT systems [9], [11], [18], [19], [22], [25], [28], [29], some of which are being used in industrial applications with millions of users, e.g., Jupiter OT [11] is the basis for Google Wave OT [29], [19] and Google Docs also uses OT for consistency maintenance; COT [25] is used in Codoxword and IBM OpenCoWeb. As OT is increasingly applied to a wider range of real-world applications and used by more and more people, ensuring and verifying correctness of the core OT algorithms become more and more important.

Driven by curiosity, we set out to study seven OT systems, including Jupiter⁶ [11], NICE [18], Google Wave OT [19], [29], SOCT3 and SOCT4 [28], TIBOT [9], and COT [25], that were designed for different purposes with different algorithms and protocols, but all claimed for or were regarded as being capable of avoiding CP2. The original objectives were to check whether these systems indeed meet the correct CP2-avoidance condition, and to find out how and why they did it (if they can). The actual scope and outcomes of this study have gone beyond the initial objectives. One major and unexpected outcome of this study is the discovery of new CP2-avoidance conditions and transformation patterns that underly a wide range of OT systems. The discovered general conditions and patterns have not only enabled us to validate these seven existing OT systems, but also inspired us to design two new OT systems capable of avoiding CP2 with a unique combination of novel features. We report these research findings in this paper.

The rest of this paper is organized as follows. First, we introduce background knowledge related to this work. Then, we discuss CP2-avoidance conditions. Next, we present basic CP2-avoidance transformation patterns and mechanisms. Based on these patterns and mechanisms, we present our CP2-avoidance validation of seven systems. Furthermore, we present the design of two new OT systems. Finally, we summarize the main contributions of this work and discuss future directions.

2 BACKGROUND KNOWLEDGE

2.1 Causal and Total Ordering Relations

Following Lamport [7], we define the causal ordering relation among editing operations as follows [4].

Definition 1 (Causal Relation “ $\rightarrow$ ”). Given two operations O_a and O_b , generated at site i and j , respectively, O_a is causally before O_b , denoted by $O_a \rightarrow O_b$, if and only if: (1) $i = j$ and the generation of O_a happened before the generation of O_b ; (2) $i \neq j$ and the execution of O_a at site j happened before the

generation of O_b ; or (3) there exists an operation O_x such that $O_a \rightarrow O_x$ and $O_x \rightarrow O_b$.

Definition 2 (Concurrent Relation “ $\parallel$ ”). Given two operations O_a and O_b , O_a and O_b are concurrent, denoted by $O_a \parallel O_b$, if and only if neither $O_a \rightarrow O_b$ nor $O_b \rightarrow O_a$.

In real-time collaborative editing systems, operations with causal relationships are executed in causal orders; but concurrent operations may be executed in any orders [4], [21]. In OT systems, concurrent operations are transformed to achieve consistency, but transformation orders are also governed by causal relations among operations.

Definition 3 (Total Ordering Relation “ $\Rightarrow$ ”). A relation “ $\Rightarrow$ ” is a total ordering relation among operations in a session if the following statements hold for any operations O_a , O_b , and O_c , where $O_a \neq O_b \neq O_c$: (1) if $O_a \Rightarrow O_b$ and $O_b \Rightarrow O_c$, then $O_a \Rightarrow O_c$ (transitivity); and (2) $O_a \Rightarrow O_b$ or $O_b \Rightarrow O_a$ (totality).

In contrast to the causal relation, which defines a partial ordering relation among operations, a total ordering (TO) relation is a complete ordering relation among operations. The “totality” condition in the total ordering relation means that all operations are ordered under “ $\Rightarrow$ ”. In this work, we are particularly interested in a special class of total ordering relations that are consistent with the causal ordering relation among the same group of operations, i.e., for any O_a and O_b , if $O_a \rightarrow O_b$, then $O_a \Rightarrow O_b$. In the rest of this paper, we assume “ $\Rightarrow$ ” is always consistent with “ $\rightarrow$ ”. As will become clear later, such total ordering relations play a key role in defining CP2-avoidance conditions.

2.2 Operation Context and Conditions

In OT systems, every operation O is associated with a context, which represents the document state (DS) on which O is defined [21], [22], [25]. For the purpose of OT design, the context of O needs not be represented by a real document state, but by a set of operations (in their original forms) that have been executed (after transformations) to create the state on which O is defined [25].

Definition 4 (Document State Representation). A document state s , denoted as $C(s)$, can be represented as follows: (1) the initial document state s_0 is represented as an empty set $C(s_0) = \{\}$; (2) after executing an operation O on the document s , the new state $s' = s \circ O$ is represented by: $C(s') = C(s) \cup \{org(O)\}$, where $org(O)$ represents the original form of O ; if O is an original operation (generated by a user), $org(O) = O$.

Definition 5 (Operation Context Representation). The context of an operation O , denoted as $C(O)$, is represented as follows: (1) for an original operation O , $C(O) = C(s)$, where s is the document state from which O is generated; (2) for a transformed operation O' , $C(O') = C(O) \cup \{org(O_x)\}$, where $O' = T(O, O_x)$, and T is a function that transforms O into O' according to the impact of O_x .

6. When Jupiter OT was published [11], CP2 had not yet been identified as a convergence requirement. So Jupiter was not designed to purposely avoid CP2, but to rely solely on CP1 for convergence.

The significance of operation context is that it provides the ground for interpreting the effect of an operation and

reasoning about the relations among operations, which are essential for ensuring correct operation execution and transformation [21], [22], [25]. Among others, two context-based conditions are directly relevant to this work:

1. *Context-equivalence condition for execution.* Given an operation O and a document state s , O can be correctly executed on s only if $C(O) = C(s)$.
2. *Context-equivalence condition for transformation.* Given two operations O_a and O_b , they can be correctly transformed with each other, by invoking $T(O_a, O_b)$ or $T(O_b, O_a)$, only if $C(O_a) = C(O_b)$.

In theory, any operation (either original or transformed) has a context, but operations included in a context are all in original forms. Causal and total ordering relations are defined only among original operations [25]. For notation conciseness, however, we assume an automatic operation form conversion, from “ O ” to “ $org(O)$ ”, whenever “ $\rightarrow$ ”, “ $\parallel$ ”, “ $\Rightarrow$ ” and other context set operators (e.g., “ $\cup$ ”) are used, e.g., “ $O_a \Rightarrow O_b$ ” means “ $org(O_a) \Rightarrow org(O_b)$ ”. When we say an operation O is in an operation context, we mean $org(O)$ is in the context. We use the notation $O_a\{b\}$ to denote that O_b is in the context of O_a , which can be used to represent the result of transforming O_a against O_b , i.e., $O_a\{b\} = T(O_a, O_b)$. If O is generated from an initial document state of a session, no operation is included in its context, so it can be expressed as $O\{\}$ or simply O .

2.3 Convergence Properties

Two transformation properties are directly related to achieving convergence in OT systems.

Convergence Property 1 (CP1). Given O_a and O_b defined on the document state s , and suppose $O_a\{b\} = T(O_a, O_b)$, and $O_b\{a\} = T(O_b, O_a)$, T preserves CP1 if:

$$s \circ O_a \circ O_b\{a\} = s \circ O_b \circ O_a\{b\},$$

which means the same document is produced by applying O_a and $O_b\{a\}$ in sequence on s , and applying O_b and $O_a\{b\}$ in sequence on s , respectively.

Convergence Property 2 (CP2). Given O_a, O_b and O_c defined on the same state, and suppose $O_a\{b\} = T(O_a, O_b)$ and $O_b\{a\} = T(O_b, O_a)$, $O_c\{a\} = T(O_c, O_a)$, and $O_c\{b\} = T(O_c, O_b)$, T preserves CP2 if:

$$T(O_c\{a\}, O_b\{a\}) = T(O_c\{b\}, O_a\{b\}),$$

which means the same operation is produced by transforming O_c against O_a and then $O_b\{a\}$, and transforming O_c against O_b and then $O_a\{b\}$, respectively.

In OT systems that allow concurrent operations to be transformed and executed in arbitrary orders, convergence can be achieved if and only if transformation functions preserve both CP1 and CP2 [15], [24], [30]. Consider a scenario with three concurrent operations O_a, O_b , and O_c generated, propagated, transformed, and executed at three sites, as shown in Fig. 1. At Site 1 and Site 2:

1. after executing O_a and O_b in different orders, their combined effects on the initial state s , i.e., $s \circ O_a \circ$

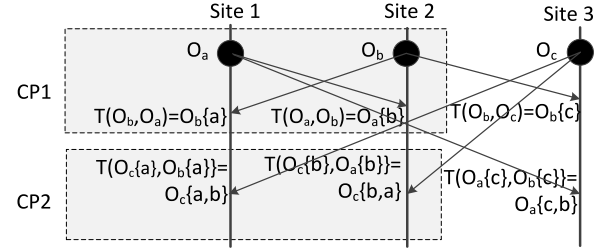


Fig. 1. An example scenario for illustrating CP1 and CP2.

$O_b\{a\}$ and $s \circ O_b \circ O_a\{b\}$, will be the same if T preserves CP1;

2. after transforming O_c with O_a and O_b in different orders, the transformation results, i.e., $T(O_c\{a\}, O_b\{a\})$ and $T(O_c\{b\}, O_a\{b\})$, will be identical if T preserves CP2.

So, the final states at Sites 1 and 2 must be convergent if T preserves CP1 and CP2. It can be shown the same final state can be achieved at Site 3 by examining all transformation and execution sequences in this scenario [24].

Designing transformation functions for preserving CP1 is relatively easy, but preserving CP2 is non-trivial [22], [27]. Under the scenario in Fig. 1, instantiated with simple character-wise insert and delete operations and a string of chars (the document state), a CP2-violation case, named the *False-Tie* (FT) puzzle, had been discovered [22], [24], [27] and often discussed in OT literature [6], [8], [13], [17], [24]. Though CP2-violation can be resolved by suitably designed functions, such a CP2-preserving solution is application-specific and different solutions have to be invented for different applications. So, many OT systems have taken the CP2-avoidance approach based on control algorithms, which are applicable to a variety of applications.

3 CP2-AVOIDANCE CONDITIONS

3.1 A General CP2-Avoidance Condition

A general CP2-avoidance condition was discovered in [25], which can be stated as: *CP2 can be avoided if an OT system can avoid transforming the same pair of operations in different contexts.* This condition requires the contexts of any two operations are always the same whenever they are transformed with each other, but it does not specify what concrete conditions the contexts of two operations should meet when they are transformed. This CP2-avoidance condition is general but unsuitable for direct use in validating specific OT systems.

3.2 A New CP2-Avoidance Condition

In this work, we have discovered a new CP2-avoidance condition that satisfies the general condition and also defines concrete context conditions between two operations for avoiding CP2. This new condition provides the ground to not only validate existing OT systems, but also design new OT systems capable of avoiding CP2 with novel features unavailable in existing systems.

Definition 6 (Totally Ordered and Equivalent Context Relation “ $\cong$ ”). Given a “ $\Rightarrow$ ” among all operations in a session, two operations O_a and O_b have a Totally Ordered and

Equivalent Context Relation, denoted by $C(O_a) \stackrel{\cong}{=} C(O_b)$, under the following conditions: (1) $O_a \Rightarrow O_b$; (2) $C(O_a) = C(O_b)$; and (3) O_x is included in $C(O_a)$ and $C(O_b)$ if and only if:

1. $O_x \Rightarrow O_a \Rightarrow O_b$; or
2. $O_a \Rightarrow O_x \Rightarrow O_b$ and $O_x \rightarrow O_b$.

The “ $\stackrel{\cong}{=}$ ” relation requires the contexts of O_a and O_b include all and only those operations that are totally before both O_a and O_b , and those that are totally between O_a and O_b and causally before O_b . The following theorem establishes the uniqueness of the context under this relation.

Theorem 1. *For any two operations O_a and O_b in a session, operations included in their contexts are uniquely defined if $C(O_a) \stackrel{\cong}{=} C(O_b)$.*

Proof. In a given session, the causal ordering relation “ $\rightarrow$ ” between any pair of operations must be fixed; and, under a given total ordering relation “ $\Rightarrow$ ”, the total ordering among all operations must be unique by Definition 3. The theorem follows directly from the uniqueness of the “ $\Rightarrow$ ” and “ $\rightarrow$ ” relations among operations in the same session and the definition of “ $\stackrel{\cong}{=}$ ”. $\square$

The general CP2-avoidance condition can be restated as follows: CP2 can be avoided if an OT system can ensure the same pair of operations are transformed always on the same context. This restatement highlights the context uniqueness essence of CP2-avoidance. The following theorem establishes a new CP2-avoidance condition.

Theorem 2. *CP2 can be avoided if an OT system ensures two operations O_a and O_b are transformed only if $C(O_a) \stackrel{\cong}{=} C(O_b)$.*

Proof. It is derived directly from the general CP2-avoidance condition and the uniqueness of the context of O_a and O_b under $C(O_a) \stackrel{\cong}{=} C(O_b)$ (Theorem 1). $\square$

4 CP2-AVOIDANCE MECHANISMS AND PATTERNS

4.1 Concurrency and Context Relation Detection

Concurrency and context relations among operations are essential for correct transformation in OT systems [25] and also a key component in the CP2-avoidance condition.

Vector logical clocks have been commonly used to capture concurrent relations among events in distributed systems [16]. Following general distributed computing, OT system designers have invented *state-vectors* [4] and *context-vectors* [25] to capture and detect the concurrency and context relationships among operations in collaborative editing systems. Such vector-based schemes have been adopted in OT systems like COT and SOCT3.

OT system designers have also invented non-vector-based schemes for detecting concurrency and context relation among operations. Those schemes use special protocols and conditions imposed on operation propagation, ordering, and buffering, which allow derivation of operation context or concurrency relations, without vector-based time-stamping. Such schemes are used in Jupiter, NICE, Google Wave OT, TIBOT and SOCT4.

4.2 Total Ordering Schemes

A total ordering scheme is another key component to achieving CP2-avoidance in OT systems. For a group of n operations in a session, there may be $n!$ possible ways of totally ordering them. The CP2-avoidance condition only requires the same total order “ $\Rightarrow$ ” is used in the same session, without imposing any constraints on specific total ordering relations or on ways of creating such total ordering relations. We have identified three types of total ordering scheme used in existing OT systems.

The first is an *implicit* total ordering scheme based on a central server: each site connects and sends local operations to the server via a FIFO (First In First Out) communication channel (e.g., a TCP connection); and the central server serializes operations and broadcasts them among all sites. The *serialization* order at the server *implicitly* forms a total order among all operations. This kind of total ordering scheme can be found in Jupiter [11], Google Wave OT [29], NICE [18], and Codoxword [25].

The second is an *explicit* total ordering scheme based on a special sequencer, which does not broadcast operations but only issues continuous sequence numbers for ordering operations. After generating a local operation, a collaborating site requests the central sequencer for a sequence number to order this operation. This total ordering scheme is used in SOCT3 and SOCT4 [28].

The third one is a *distributed* total ordering scheme without involving any central server/sequencer: collaborating sites use special rules to broadcast and timestamp operations so that a total ordering relation among all operations can be unambiguously derived. One example of such distributed schemes is TIBOT [9].

One way or another, all existing OT systems that claimed or were recognized for CP2-avoidance have a total ordering scheme. However, the formation and use of such total ordering may be implicit in those systems; and the intricate links between the total ordering and CP2-avoidance were generally unknown to most system designers. Discovering the hidden role of a total ordering scheme is a critical step towards discovering the general CP2-avoidance condition and patterns in existing OT systems. Finally, it is worth pointing out that in all existing OT systems capable of avoiding CP2 (see Sections 5 and 6), the total ordering is used for controlling the order of executing remote operations and transforming concurrent operations; local operations can be freely generated and immediately executed without being constrained by the total ordering.

4.3 Operation and Transformation Patterns

In this work, we have also discovered basic operation sequences and transformation patterns that can ensure $C(O_a) \stackrel{\cong}{=} C(O_b)$ in transformations, as described below.

Definition 7 (Totally-Ordered and Contextualized Operation Sequence (TOCOS)). *Given a list of operations $L = [O_1, O_2, \dots, O_n]$ and a “ $\Rightarrow$ ” among all operations in a session, L is a TOCOS if it is an empty list or a list with one operation, or the following holds (when $n \geq 2$): (1) $O_i \Rightarrow O_j$, for $1 \leq i < j \leq n$; and (2) $C(O_{i+1}) = C(O_i) \cup \{O_i\}$, for $1 \leq i < n$.*

All operations (if any) in a TOCOS are required to be totally ordered and contextualized, i.e., the context of an operation O_{i+1} must include O_i and all operations in the context of O_i . Contextualization can be achieved at the time of operation generation if $O_i \rightarrow O_{i+1}$, or by transformation if they are concurrent, i.e., $O_i || O_{i+1}$.

Based on TOCOS, we define two basic operation sequence patterns related to avoiding CP2.

Definition 8 (Operation Sequence Pattern 1 (OSP1)). Given a TOCOS $L = [O_1, O_2, \dots, O_n]$ and an operation O_x that is not in L , L is an OSP1 with respect to O_x if L is an empty list or the following conditions hold: (1) $C(O_1) \supseteq C(O_x)$; (2) $O_n \Rightarrow O_x$; and (3) L includes every operation O_y in a session, such that $O_y || O_x$ and $O_1 \Rightarrow O_y \Rightarrow O_n$ (when $n \geq 2$).

The first operation (if any) in L must have the " $\supseteq$ " relation with O_x ; all operations in L must be totally ordered before O_x ; all operations that are concurrent with O_x and totally ordered between O_1 and O_n must be included in L .

Definition 9 (Operation Sequence Pattern 2 (OSP2)). Given a TOCOS $L = [O_1, O_2, \dots, O_n]$ and an operation O_x that is not in L , L is an OSP2 with respect to O_x if L is an empty list or the following conditions hold: (1) $C(O_x) \supseteq C(O_1)$; (2) $O_i \rightarrow O_j$, for $1 \leq i < j \leq n$ (when $n \geq 2$); and (3) L includes every operation O_y in the session, such that $O_1 \rightarrow O_y \rightarrow O_n$.

O_x must have the " $\supseteq$ " relation with the first operation (if any) in L , which implies O_x is totally ordered before all operations in L ; all operations in L must be causally related; and all operations that are causally ordered between O_1 and O_n must be included in L .

The following LT function transforms an operation O against a sequence of operations in L to produce $O\{L\}$.

List Transformation: $LT(O, L) = O\{L\}$

1. $O\{L\} := O$;
2. **for** ($i = 1; i \leq |L|; i++$) {
 $O\{L\} := T(O\{L\}, L[i]);$ }

The SLT function symmetrically transforms an operation O with operations in L to produces $O\{L\}$ and $L\{O\}$.

Symmetric List Transformation: $SLT(O, L) = (O\{L\}, L\{O\})$

1. $O\{L\} := O$;
2. **for** ($i = 1; i \leq |L|; i++$) {
 $L\{O\}[i] := T(L[i], O\{L\});$
 $O\{L\} := T(O\{L\}, L[i]);$ }

$LT(O_x, L)$ and $SLT(O_x, L)$ define two basic CP2-avoidance transformation patterns under the condition that L is an OSP1 or OSP2 with respect to O_x , which is established by the following theorem.

Theorem 3. If L is an OSP1 or OSP2 with respect to O_x , then CP2 is avoided in $LT(O_x, L)$ and $SLT(O_x, L)$.

Proof. Consider the transformation pattern: $LT(O_x, L)$ where L is an OSP1 with respect to O_x . From Definition 8, we know $C(L[1]) \supseteq C(O_x)$. Therefore, CP2 can be avoided in $T(O_x, L[1])$ by Theorem 2. After this transformation, O_x becomes $O_x\{L[1]\}$, which implies $C(L[2]) \supseteq C(O_x\{L[1]\})$ by Definitions 6 and 8, hence CP2 can be avoided in

$T(O_x\{L[1]\}, L[2])$. By an inductive argument, CP2 can be avoided in every transformation in $LT(O_x, L)$. Following the same reasoning, CP2 can be avoided in all transformation patterns stated in the theorem. $\square$

As will be shown later in this paper, OSP1 and OSP2 provide effective tools for validating CP2-avoidance of existing OT systems and guiding the design of new OT systems. They also possess interesting properties, established in the following theorems, that are useful in designing efficient CP2-avoiding OT systems (Section 6.2).

Theorem 4. Let L be an OSP1 with respect to two different operations O_x and O_y in the same session. Under the CP2-avoidance condition, an operation in L may be transformed with both O_x and O_y under the same context.

Proof. Since L is an OSP1 with respect to both O_x and O_y , we have $C(L[1]) \supseteq C(O_x)$ and $C(L[1]) \supseteq C(O_y)$, by Definition 8 (OSP1). Since all operations in the same session are totally ordered under " $\Rightarrow$ ", we can assume $O_x \Rightarrow O_y$ and derive $L[1] \Rightarrow O_x \Rightarrow O_y$ (follow the same reasoning if $L[1] \Rightarrow O_y \Rightarrow O_x$). Under $L[1] \Rightarrow O_x \Rightarrow O_y$ and the CP2-avoidance condition (Theorem 2), $L[1]$ is permitted to be transformed with both O_x and O_y under the same context $C(L[1])$. By an inductive argument, every operation in L may be transformed with two *totally-after* operations under the same context. $\square$

Theorem 5. Let L be an OSP2 with respect to two different operations O_x and O_y in the same session. Under the CP2-avoidance condition, no operation in L will be transformed with O_x and O_y under the same context.

Proof. Since L is an OSP2 with respect to both O_x and O_y , we have $C(O_x) \supseteq C(L[1])$ and $C(O_y) \supseteq C(L[1])$, by Definition 9 (OSP2). Since all operations in the same session are totally ordered under " $\Rightarrow$ ", we can assume $O_x \Rightarrow O_y$ and derive $O_x \Rightarrow O_y \Rightarrow L[1]$ (the same reasoning can be applied for $O_y \Rightarrow O_x \Rightarrow L[1]$). Under $O_x \Rightarrow O_y \Rightarrow L[1]$ and the CP2-avoidance condition, $L[1]$ is permitted to be transformed with O_x , but not O_y , under the context $C(L[1])$. This is because for $L[1]$ and O_y to be transformed with each other, their contexts must include O_x due to $O_x \Rightarrow O_y \Rightarrow L[1]$, but $C(L[1])$ cannot include O_x since $C(L[1]) = C(O_x)$. Therefore, $L[1]$ cannot be transformed with both O_x and O_y under the same context $C(L[1])$. By induction, no operation in L can be transformed with two *totally-before* operations under the same context. $\square$

5 VALIDATING CP2-AVOIDANCE IN OT SYSTEMS

5.1 A Framework for Dealing with Diversity

One major challenge in validating existing OT systems is the diversity of those systems to be examined. They were designed for different purposes and had major differences in: (1) system structures, e.g., some use a transformation-based central server and some do not; (2) operation propagation protocols, e.g., some delay propagating operations until certain conditions are met, and some do not; and (3) mechanisms and algorithms, e.g., some use state/

context-vectors, and some do not; some are based on two-dimensional (2D) state-spaces, and some use one-dimensional (1D) buffers. These systems were designed by different research groups and described in different styles and terminologies, which present challenges to figure out their real similarities and differences.

Our approach to addressing these challenges is to devise a general framework that encapsulates the basic mechanisms and transformation patterns that may occur at three critical locations: (1) local sites, where operations are generated by users; (2) the server site (optional), where operations are broadcast to other sites; and (3) remote sites, where operations generated by other sites are replayed. Then, we dissect the seven OT systems one-by-one to extract relevant schemes and describe them uniformly in terms of the basic mechanisms and patterns under the framework. Finally, we validate CP2-avoidance by matching their transformation patterns with CP2-avoidance patterns established in this work.

We use a running example to illustrate how a system works in concrete scenarios, in which there are three collaborating sites (plus an optional server) and four operations with causal relations: $(O_1 \rightarrow O_4) \parallel (O_2 \rightarrow O_3)$, and total ordering relations: $O_1 \Rightarrow O_2 \Rightarrow O_3 \Rightarrow O_4$.

5.2 Jupiter

In the Jupiter system [11], collaborating clients are connected to a central server via TCP connections in a star-like topology. A client can communicate directly with the server only; the server sequentially processes and broadcasts operations among all clients. The server is also involved in transforming operations.

5.2.1 Local Processing

When O_x is generated by a client, it is executed immediately at the local site and propagated to the server as-is. Local operations are propagated sequentially and saved along the local dimension of a 2D state-space.

5.2.2 Server Processing

The server maintains multiple 2D state-spaces,⁷ one for every client. A state-space consists of a *local* dimension for operations from the corresponding client, and a *global* dimension for operations from all other sites. The server handles O_x as follows:

1. locates the space for the client from which O_x comes;
2. searches this space to find the state that matches the context of O_x , and saves O_x at this matching state along the local dimension;
3. transforms O_x symmetrically with a sequence of operations, denoted as L_1 , consisting of operations along the global dimension from the matching state to the final state of the space; this transformation can be expressed as: $SLT(O_x, L_1) = (O_x\{L_1\}, L_1\{O_x\})$;
4. propagates $O_x\{L_1\}$ to other clients;

7. The multiple 2D state-spaces maintenance was not reported in the Jupiter paper [11]. The description here is based on our derivation from other sources [10], [19], [29].

5. saves $O_x\{L_1\}$ at the end of the global dimension of every other state-space maintained by the server.

5.2.3 Remote Processing

Each client maintains a state-space similar to that at the server. Jupiter handles $O_x\{L_1\}$ at a remote site as follows:

1. searches the space to find a matching state for $O_x\{L_1\}$, and saves $O_x\{L_1\}$ at this matching state along the global dimension;
2. transforms $O_x\{L_1\}$ symmetrically with a sequence of operations, denoted as L_2 , consisting of operations along the local dimension from the matching state to the final state of the space; this can be expressed as $SLT(O_x\{L_1\}, L_2) = (O_x\{L_1, L_2\}, L_2\{O_x\})$;
3. executes $O_x\{L_1, L_2\}$.

5.2.4 CP2-Avoidance Validation

It can be shown that L_1 is an OSP1 with respect to O_x for $SLT(O_x, L_1)$, and L_2 is an OSP2 with respect to $O_x\{L_1\}$ for $SLT(O_x\{L_1\}, L_2)$ because:

1. all operations in L_1 and L_2 are:
 - 1) totally ordered by the server serialization;
 - 2) contextualized by transformations in SLT ;
 - 3) concurrent with O_x as they are generated by other clients before O_x reaches these clients, and O_x is generated before they reach O'_x 's local site;
2. L_1 contains all operations that are totally before O_x , and $L_1[1] \stackrel{\Rightarrow}{=} O_x$;
3. L_2 contains operations that are totally after O_x and causally related with each other, and $O_x\{L_1\} \stackrel{\Rightarrow}{=} L_2[1]$.

By Theorem 3, Jupiter CP2-avoidance is confirmed. The reader is referred to [30] for detailed illustration of the working of Jupiter in the running example.

5.3 NICE

The NICE system [18] is also based on a central server for serializing, broadcasting, and transforming operations. NICE supports OT-based flexible notification for both real-time and non-real-time collaboration. We describe only the relevant consistency maintenance aspects below.

5.3.1 Local Processing

NICE handles local operations in the same way as Jupiter, except using a 1D buffer to save local operations only.

5.3.2 Server Processing

The server maintains multiple 1D buffers, one for every client to maintain operations that are from other clients. The server handles O_x as follows:

1. locates the buffer for the client from which O_x comes;
2. searches the buffer to find the matching state for O_x ;
3. transforms O_x symmetrically with a sequence of operations, denoted as L_1 , starting from matching state to the end of the buffer; this transformation can be expressed as: $SLT(O_x, L_1) = (O_x\{L_1\}, L_1\{O_x\})$;
4. propagates $O_x\{L_1\}$ to other clients;
5. saves $O_x\{L_1\}$ at the end of every other 1D buffer.

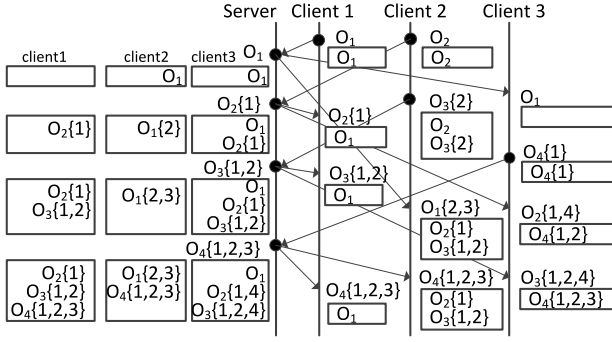


Fig. 2. NICE: The server maintains three 1D buffers; each client maintains one buffer.

5.3.3 Remote Processing

NICE handles $O_x\{L_1\}$ at a remote site as follows:

1. searches the buffer for the state that matches the context of $O_x\{L_1\}$ (this state may not exist);
2. transforms $O_x\{L_1\}$ symmetrically with a sequence of operations, denoted as L_2 , consisting of operations from the matching state (if exists) to the end of the buffer; this transformation can be expressed as $SLT(O_x\{L_1\}, L_2) = (O_x\{L_1, L_2\}, L_2\{O_x\})$;
3. executes $O_x\{L_1, L_2\}$ (no need to save it in the buffer).

5.3.4 CP2-Avoidance Validation

It can be shown that L_1 is an OSP1 with respect to O_x in $SLT(O_x, L_1)$; and L_2 is an OSP2 with respect to $O_x\{L_1\}$ in $SLT(O_x\{L_1\}, L_2)$ by following the same reasoning for Jupiter. By Theorem 3, NICE CP2-avoidance is confirmed.

5.3.5 An Example

Consider operation $O_3\{2\}$ at Client 2 in Fig. 2. Upon generation, it is executed immediately at Client 2, appended at the end of the local buffer, and propagated to the server. At the server, the buffer for Client 2 is located and searched to find the matching state at beginning of the buffer. Then, $O_3\{2\}$ is symmetrically transformed with $O_1\{2\}$, which produces two transformed operations: (1) $O_3\{1,2\}$ is propagated to Clients 1 and 3, and appended to the ends of their buffers as well; (2) $O_1\{2,3\}$ is used to replace $O_1\{2\}$ in the buffer for Client 2. When $O_3\{1,2\}$ arrives at Client 3, it is symmetrically transformed with $O_4\{1,2\}$ since they share the same state, which produces two transformed operations: $O_3\{1,2,4\}$ is executed; and $O_4\{1,2,3\}$ is used to replace $O_4\{1,2\}$ in the buffer.

5.4 Google Wave OT

Google Wave OT [10], [19], [29] is adapted from Jupiter, with the same star-like architecture and a central server for broadcasting and transforming operations, but adds a *stop-and-wait* protocol between a client and the server. As a result, a single 1D buffer at the server is adequate to maintain all transformation states, which eliminates multiple 2D state-spaces in Jupiter.

5.4.1 Local Processing

When O_x is generated, it is executed immediately and saved in the local dimension of a 2D state-space. However, O_x is not propagated to the server until the prior local operation O_{x-1} (if any) has been acknowledged by the server. Consequently, when O_x is propagated, it must have been symmetrically transformed with a sequence of remote operations (if any), denoted as L_1 , that are propagated from the server (see the follow-up description for remote processing) and totally before O_{x-1} (hence before O_x as well). This transformation can be expressed as: $SLT(O_x, L_1) = (O_x\{L_1\}, L_1\{O_x\})$; $O_x\{L_1\}$ is propagated to the server.

5.4.2 Server Processing

The server maintains one 1D buffer for keeping track operations from all clients in a totally ordered and contextualized fashion. The server handles $O_x\{L_1\}$ as follows:

1. searches the 1D buffer to find the state that matches the context of $O_x\{L_1\}$;
2. transforms $O_x\{L_1\}$ against a sequence of operations, denoted as L_2 , from the matching state to the end of the buffer; this transformation can be expressed as $LT(O_x\{L_1\}, L_2) = O_x\{L_1, L_2\}$;
3. appends $O_x\{L_1, L_2\}$ at the end of the buffer (other operations in the buffer remain unchanged); and
4. propagates $O_x\{L_1, L_2\}$ to all sites, including the originating site of $O_x\{L_1\}$ as an acknowledgement.

5.4.3 Remote Processing

At each client site, there is a 2D state-space similar to Jupiter. The handling of $O_x\{L_1, L_2\}$ at a remote site is also similar to handling $O_x\{L_1\}$ in Jupiter: $O_x\{L_1, L_2\}$ will be symmetrically transformed with a sequence of operations, denoted as L_3 , consisting of operations along the local dimension from the state that matches the context of $O_x\{L_1, L_2\}$ to the final state of the state-space. This transformation can be expressed as $SLT(O_x\{L_1, L_2\}, L_3) = (O_x\{L_1, L_2, L_3\}, L_3\{O_x\})$. Finally, $O_x\{L_1, L_2, L_3\}$ is executed.

5.4.4 CP2-Avoidance Validation

It can be shown that L_1 is an OSP1 with respect to O_x in $SLT(O_x, L_1)$, L_2 is an OSP1 with respect to $O_x\{L_1\}$ in $LT(O_x\{L_1\}, L_2)$, and L_3 is an OSP2 for $O_x\{L_1, L_2\}$ in $SLT(O_x\{L_1, L_2\}, L_3)$ by following the same reasoning as for Jupiter. Google Wave OT CP2-avoidance is confirmed.

5.4.5 An Example

Consider operation $O_3\{2\}$ at Client 2 in Fig. 3. Upon generation and execution, it is appended at the end of the local dimension of a 2D state-space. It is not propagated until it has been transformed with O_1 and Client 2 has received the acknowledgement of $O_2\{1\}$, which is the prior operation causally before $O_3\{2\}$. The transformed operation $O_3\{1,2\}$ is propagated to the server. At the server, the matching state is located at the end of $O_2\{1\}$ in the buffer and there is no other operation shares this state, so $O_3\{1,2\}$ is appended to

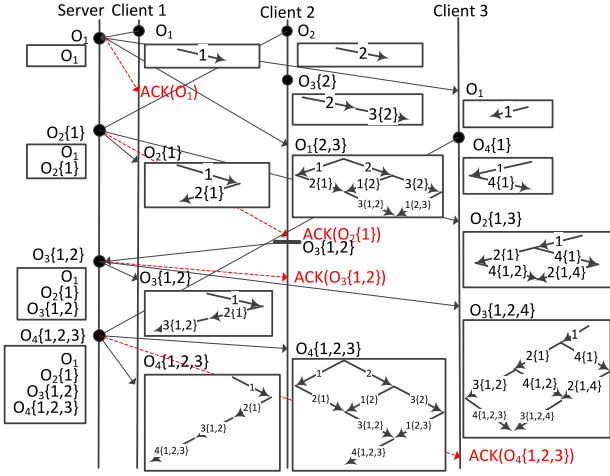


Fig. 3. Google Wave OT: The server maintains a single linear buffer; each client maintains one 2D state-space; and local operations are acknowledged by the server. In a 2D state-space, an operation $O_n\{m\}$ is represented by arrow labeled by $n\{m\}$ (for conciseness), the local dimension is represented by south-east arrows, and the global dimension represented by south-west arrows.

the end of the buffer and then propagated to all clients. When $O_3\{1,2\}$ arrives at Client 3, it is handled in exactly the same way as in Jupiter.

5.5 SOCT4 and SOCT3

SOCT4 and SOCT3 are closely related to each other. Unlike prior systems, SOCT 4 and SOCT 3 use a central sequencer, which issues *continuous* sequence numbers to totally order operations, but is not involved in transforming or broadcasting operations. SOCT4 imposes a *global serialization* on operation propagation. SOCT3 uses additional state-vectors to capture operation concurrency relations. In this paper, we focus on SOCT4 only; the reader is referred to [30] for description and validation of SOCT3.

5.5.1 Local Processing

When O_x is generated, SOCT4 handles it as follows:

1. executes O_x immediately and acquires a sequence number from the central sequencer to timestamp O_x ;
2. saves O_x in *HB*, and waits until O_x has been symmetrically transformed with a sequence of remote operations, denoted as L_1 , that are totally before O_x (i.e., globally serialized propagation); this can be expressed as: $SLT(O_x, L_1) = (O_x\{L_1\}, L_1\{O_x\})$;
3. propagates $O_x\{L_1\}$.

5.5.2 Remote Processing

SOCT4 handles $O_x\{L_1\}$ at a remote site as follows:

1. waits until all operations that are totally before $O_x\{L_1\}$ have been executed at this site;
2. transforms $O_x\{L_1\}$ symmetrically with a sequence of local operations that are totally after it in *HB*, denoted as L_2 ; this transformation can be expressed as: $SLT(O_x\{L_1\}, L_2) = (O_x\{L_1, L_2\}, L_2\{O_x\})$;
3. executes $O_x\{L_1, L_2\}$ (no need to save it in the buffer).

5.5.3 CP2-Avoidance Validation

It can be shown that L_1 is an OSP1 with respect to O_x in $SLT(O_x, L_1)$, and L_2 is an OSP2 with respect to $O_x\{L_1\}$ in $SLT(O_x\{L_1\}, L_2)$ by the same reasoning as for Jupiter. By Theorem 3, SOCT4 CP2-avoidance is confirmed.

The reader is referred to [30] for detailed illustration of the working of SOCT4 in the running example.

5.6 TIBOT

TIBOT (Time-Interval Based OT) is unique in using a distributed scheme to totally order operations, without using any central server. Collaborating sites may communicate in any way that ensures reliability and the FIFO property.

5.6.1 Time Interval-Based Total Ordering

Each site maintains a local timer (a numeric counter) with continuously increasing *Time-Interval (TI)* values: 0, 1, 2, ..., etc. Timers at different sites must take the same sequence of values, but may tick at different speeds and need not be synchronized. When an operation is generated at a site, it is timestamped by a tuple $\langle TI, SI, SN \rangle$, where TI is the local timer value, SI is the local site identifier, SN is the local operation sequence number.

TI-based total ordering scheme. For any two operations O_a and O_b , with timestamps T_a and T_b , respectively, $O_a \Rightarrow O_b$ if and only if: (1) $T_a.TI < T_b.TI$; or (2) $T_a.TI = T_b.TI$ and $T_a.SI < T_b.SI$; or (3) $T_a.TI = T_b.TI$ and $T_a.SI = T_b.SI$ and $T_a.SN < T_b.SN$.

5.6.2 Local Processing

When O_x is generated, TIBOT handles it as follows:

1. executes O_x immediately and timestamps it with T_x ;
2. saves O_x in a history buffer (*HB*) and waits until: (1) the local timer has been advanced to a value larger than $T_x.TI$; and (2) O_x has been transformed with a sequence of remote operations, denoted as L_1 , which includes all operations whose *TIs* are smaller than $T_x.TI$ (i.e., *TI*-based synchronization in propagation); this can be expressed as $LT(O_x, L_1) = O_x\{L_1\}$;
3. propagates $O_x\{L_1\}$ with the same timestamp T_x .

There are two auxiliary operation propagation schemes. First, all local operations generated during the same time interval are propagated to remote sites together, which not only reduces communication overhead, but also simplifies derivation of total order relations at remote sites. Second, if no operation is generated at a site during a time interval, this site is required to broadcast a special message, with a timestamp $\langle TI, SI, nil \rangle$, at the end of each time interval. Such special messages are needed to enable the derivation of *TI*-based conditions and total ordering relations among operations in TIBOT.

5.6.3 Remote Processing

TIBOT handles $O_x\{L_1\}$ at a remote site as follows:

1. waits until: (1) the local timer has advanced to a value greater than $T_x.TI$; (2) local operations with *TIs* equal to $T_x.TI$ have been propagated; and (3) all

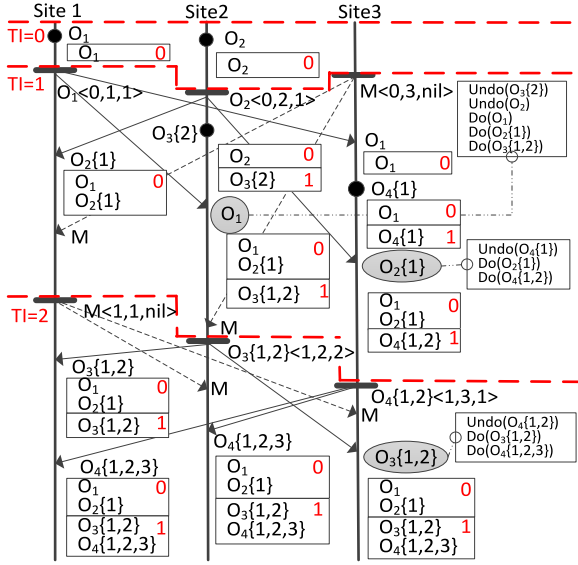


Fig. 4. TIBOT: Each site maintains an independent timer and a linear HB; operations may be undone and redone; special message M is sent if no local operation to propagate.

operations totally before $O_x\{L_1\}$ have been executed at this site;

2. undoes a sequence of (local) operations in HB, denoted as L_{undo} , which are totally after $O_x\{L_1\}$;
3. transforms $O_x\{L_1\}$ against a sequence of operations in HB, denoted as L_2 , whose TIs are the same as $T_x.TI$; this can be expressed as $LT(O_x\{L_1\}, L_2) = O_x\{L_1, L_2\}$;
4. executes $O_x\{L_1, L_2\}$ and saves it in HB;
5. transforms operations in L_{undo} symmetrically with $O_x\{L_1, L_2\}$; this transformation can be expressed as $SLT(O_x\{L_1, L_2\}, L_{undo}) = (O_x\{L_1, L_2, L_{undo}\}, L_{undo}\{O_x\{L_1, L_2\}\})$;
6. re-executes transformed operations in $L_{undo}\{O_x\{L_1, L_2\}\}$ and saves them sequentially in HB.

5.6.4 CP2-Avoidance Validation

It can be shown that L_1 is an OSP1 with respect to O_x in $LT(O_x, L_1) = O_x\{L_1\}$, L_2 is an OSP1 with respect to $O_x\{L_1\}$ in $LT(O_x\{L_1\}, L_2)$; and L_{undo} is an OSP2 with respect to $O_x\{L_1, L_2\}$ in $SLT(O_x\{L_1, L_2\}, L_{undo})$ by following the same reasoning as for Jupiter, except that the total ordering and concurrency relations among operations are detected by TI-based distributed timestamps. By Theorem 3, TIBOT CP2-avoidance is confirmed.

5.6.5 An Example

Consider operation O_2 generated during $TI = 0$ at Site 2 in Fig. 4. Upon generation, it is executed immediately, timestamped with $\langle 0, 2, 1 \rangle$, and saved at the end of the HB. O_2 is not propagated until its timer has advanced to $TI = 1$. TIBOT handles O_2 at Site 3 as follows: Step 1: local timer has advanced to $TI = 1$; there is no local operation during $TI = 0$ and a timestamp message $M \langle 0, 3, nil \rangle$ has been propagated; O_1 is the only operation totally-before O_2 and has been executed. Step 2: local operation $O_4\{1\}$ is undone as it is totally after O_2 . Step 3: O_2 is transformed with O_1 to

produce $O_2\{1\}$. Step 4: $O_2\{1\}$ is executed and saved in HB. Step 5: $O_4\{1\}$ is transformed with $O_2\{1\}$ to produce $O_4\{1, 2\}$. Step 6: $O_4\{1, 2\}$ is re-executed and saved in HB. $O_4\{1, 2\}$ is propagated when TI is advanced to 2.

5.7 COT

COT (Context-based OT) [25] is unique in supporting both consistency maintenance and *group undo* (initiated by users, not as internal concurrency control mechanisms) [12], [14], [25]. In this study, we focus on COT consistency maintenance (convergence) aspect only. For simplicity, we assume the basic COT buffering scheme, which saves operations in original forms only [25]. We sketch COT under the assumption that every collaborating site is connected to a central server via a TCP channel; the server broadcasts and serializes operations, but does not transform operations.

5.7.1 Local Processing

When an operation O_x is generated at a site, it is executed immediately and timestamped by a Context-Vector (CV) for capturing context relations among operations, and propagated as-is to remote sites (via the server). All local operations are saved in original forms in a local buffer, called *Document State*, which may be implemented as a set or other suitable data structures.

5.7.2 Server Processing

The server broadcasts O_x to all sites, including the site from which O_x comes, which serves as a notification of its total ordering determined by the server serialization.

5.7.3 Remote Processing

After all operations that are totally before O_x have been executed at a remote site, COT handles O_x as follows:

1. finds all operations in the local DS that are concurrent with O_x by using their context-vectors;
2. contextualizes those operations into a sequence of operations, denoted as L , which are totally ordered according to the server serialization;
3. transforms O_x against operations in L , which can be expressed as: $LT(O_x, L) = O_x\{L\}$;
4. executes $O_x\{L\}$;
5. saves the original O_x in the local DS.

5.7.4 CP2-Avoidance Validation

We can use the total ordering position of O_x to split L into two sublists: $L = L_1 + L_2$, where L_1 contains operations totally before O_x and L_2 contains operations totally after O_x . Then, it can be shown that L_1 is an OSP1 with respect to O_x in $LT(O_x, L_1)$, and L_2 is an OSP2 with respect to $O_x\{L_1\}$ in $LT(O_x\{L_1\}, L_2)$ by following the same reasoning as for prior systems, except that concurrency relations among operations are detected by context-vectors. By Theorem 3, COT CP2-avoidance is confirmed, which is consistent with prior verification [25]. The reader is referred to [30] for illustration of COT in the running example.

6 NEW OT SYSTEMS INSPIRED BY THIS STUDY

The basic CP2-avoidance condition and transformation patterns discovered from this study have not only enabled us to validate seven existing OT systems, but also inspired us to devise new OT systems capable of avoiding CP2 with unique combinations of novel features.

6.1 TIBOT 2.0

The original TIBOT is unique not only in its distributed total ordering scheme, but also in its capability of avoiding both CP1 and CP2, which is only achieved by the prior GOT system [22]. But TIBOT avoids both CP1 and CP2 without using state-vectors or exclusive transformation, which is an advantage over GOT. Similar to GOT, TIBOT has used an *undo/transform-do/transform-redo* scheme to enforce a totally ordered transformation among all operations and never transform a pair of operations in different orders. However, past research has found undo/redone-based schemes are inefficient and may cause interface abnormality to end-users [21], [24], so most OT systems were designed to avoid internal undo/redone. Inspired by the insights drawn from this study, we have designed a new OT system, named as TIBOT 2.0, which inherits from TIBOT the basic time-interval-based distributed total ordering and shares the same local processing and propagation control schemes, but has a significantly resigned remote processing to eliminate undo/redone. We describe TIBOT 2.0 remote processing below.

6.1.1 Remote Processing

TIBOT 2.0 handles $O_x\{L_1\}$ at a remote site as follows:

1. waits until: The same conditions as specified in Step 1 in the original TIBOT;
2. transforms $O_x\{L_1\}$ against a sequence of operations in HB , denoted as L_2 , which includes all operations with TIs equal to $T_x.TI$ and totally before $O_x\{L_1\}$; this transformation can be expressed as $LT(O_x\{L_1\}, L_2) = O_x\{L_1, L_2\}$;
3. saves $O_x\{L_1, L_2\}$ at the end of L_2 in HB ;
4. transforms $O_x\{L_1, L_2\}$ symmetrically with a sequence of operations in HB , denoted as L_3 , which are totally after $O_x\{L_1, L_2\}$; this transformation can be expressed as $SLT(O_x\{L_1, L_2\}, L_3) = (O_x\{L_1, L_2\}, L_3\{O_x\})$;
5. executes $O_x\{L_1, L_2, L_3\}$; replaces L_3 with $L_3\{O_x\}$ in HB .

The key insight in TIBOT 2.0 is (in Step 4) to transform $O_x\{L_1, L_2\}$ symmetrically with a sequence of operations in L_3 , which are totally after $O_x\{L_1, L_2\}$. This symmetric transformation replaces undoing and redoing the same sequence of operations in $L_{undo}(=L_3)$ in Steps 2 and 5 of the original TIBOT. This is the key to avoiding CP2 without undo/redone (but giving up CP1-avoidance as CP1 can be easily achieved by transformation functions).

In addition, TIBOT 2.0 made other subtle but important technical changes (due to the elimination of undo/redone):

1. Step 2: the condition for transformation of $O_x\{L_1\}$ has been enhanced to require operations in L_2 to be

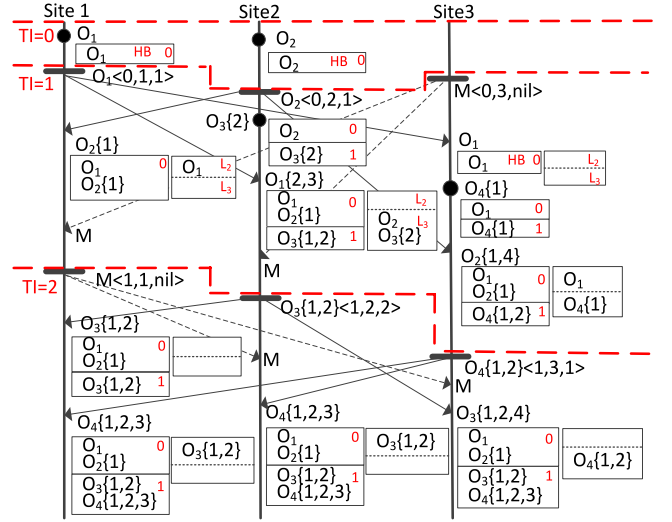


Fig. 5. TIBOT 2.0: Achieving CP2-avoidance without undo/transform-do/transform-redo or a central server.

totally before $O_x\{L_1\}$ as well as having TIs equal to T_x , which is in contrast to requiring TIs equal to T_x only in Step 3 in the original TIBOT.

2. Step 5: $O_x\{L_1, L_2, L_3\}$ is executed to achieve the original effect of O_x , which is in contrast to executing $O_x\{L_1, L_2\}$ in Step 4 and re-executing undone operations in Step 6 in the original TIBOT.

TIBOT 2.0 is unique in avoiding CP2 by distributed total ordering (no central server) without undo/redone.

It can be shown that L_2 is OSP1 with respect to $O_x\{L_1\}$ in $LT(O_x\{L_1\}, L_2)$, and L_3 is OSP2 with respect to $O_x\{L_1, L_2\}$ in $SLT(O_x\{L_1, L_2\}, L_3)$ by following the same reasoning as in validating other systems. By Theorem 3, TIBOT 2.0 CP2-avoidance is confirmed.

6.1.2 An Example

In Fig. 5, we illustrate the working of TIBOT 2.0 under the same running example. Consider O_2 at Site 2, local processing of O_2 is the same as in the original TIBOT. TIBOT 2.0 handles O_2 at Site 3, where $HB = [O_1, O_4\{1\}]$, as follows. In Step 1, the same conditions are met. In Step 2, O_2 is transformed with $L_2 = [O_1]$, which is totally before O_2 , to produce $O_2\{1\}$. In Step 3, $O_2\{1\}$ is saved in HB after O_1 . In Step 4, $O_2\{1\}$ is symmetrically transformed with $L_3 = [O_4\{1\}]$, which is totally after $O_2\{1\}$, to produce $O_2\{1,4\}$ and $[O_4\{1,2\}]$. In Step 5, $O_2\{1,4\}$ is executed at Site 3, and $O_4\{1,2\}$ is saved to replace $O_4\{1\}$ in HB . By comparing Figs. 4 with 5, we can see the undo/redone processes for O_1 , $O_2\{1\}$ and $O_3\{1,2\}$ have been eliminated, and their execution forms have also been changed due to changes in Step 5 of TIBOT 2.0.

6.2 POT: Pattern-Based OT

We have designed a brand new OT algorithm, named POT (Pattern-based OT), which possesses a unique combination of the following features:

1. works by directly following CP2-avoidance conditions and patterns discovered from this study;
2. derives operation concurrency and context relationships, without a vector-based scheme (e.g., SOCT3

- and COT), which is beneficial for large scale collaborations with an arbitrary number of dynamic users;
3. propagates local operations without imposing any restrictions (e.g., the *stop-and-wait* protocol in Google Wave OT, or *global serialization* in SOCT4, or *time-interval-based synchrolization* in TIBOT), which helps improve system robustness, and user interactive experiences;
 4. performs all transformations only at client sites, without requiring a transformation-based server (e.g., Jupiter, Google Wave OT, NICE), which simplifies the OT system structure and improves scalability;
 5. works with different total ordering schemes, with or without using a serialization server (e.g., a distributed protocol in [20]), which makes the system adaptable to both client-server and p2p environments;
 6. achieves a transformation complexity that is linear to the number of concurrent operations, which is the best in OT time efficiency; and
 7. supports both consistency maintenance (do) and user-initiated undo (like COT).

In this paper, like our treatment to COT, we focus on consistency maintenance aspect of POT only, and assume that every collaborating site is connected to a central server, which broadcasts and serializes operations, but does not transform operations.

6.2.1 Transformation Map

One key data structure in POT is the *Transformation Map* (TM), which plays a crucial role in detecting operations that are concurrent with an incoming remote operation without using state/context-vectors, and in contextualizing the detected concurrent operations without using recursive or repeated transformations [25].

Each client C_i maintains a transformation map, denoted as TM_i , which consists of $N - 1$ *transformation paths* (TP), one for every other remote client. For notation convenience, we use $TM_{i[j]}$ to denote the transformation path for remote C_j , where $i \neq j$ and $1 \leq i, j \leq N$, maintained by C_i .

$TM_{i[j]}$ contains *all* and *only* those operations that are *not* generated by remote client C_j , and represents a path for transforming operations generated by C_j . It will become clear later that a transformation path actually consists of OSP1 and OSP2 sequences that are readily usable for transforming incoming remote operations. Detailed maintenance and use of TM are explained below in the context of local and remote processing.

6.2.2 Local Processing

When an operation O_x is generated at C_i , it is handled by POT as follows:

1. executes O_x in its original form immediately at C_i ;
2. timestamps O_x with $T_x = \langle SI, RTO \rangle$, where SI is the local site identifier, and RTO is the highest TO (*Total Ordering*), assigned by the server (to be described below), among all *remote* operations that have been executed at C_i before the generation of O_x ;

3. saves O_x at the end of every transformation path $TM_{i[j]}$, $1 \leq j \leq N$, $i \neq j$, at C_i ; and
4. propagates O_x with $T_x = \langle SI, RTO \rangle$ to the server.

6.2.3 Server Processing

Operations from all clients are serialized at the server site. The server assigns a *Total Ordering* number to O_x , and then broadcasts O_x with $T_x = \langle SI, RTO, TO \rangle$ to all client sites, including the site from which O_x comes, which serves as a notification of O_x 's total ordering. It should be noted that TO may be determined with or without using a serialization server; TO should be unique for each operation, ascending, but need not to be continuous. The server does not perform transformation on operations.

6.2.4 Remote Processing

After all operations that are totally ordered before O_x , which is generated by C_i , have been executed at a remote site C_j , POT handles O_x as follows:

1. locates $TM_{j[i]}$ for the client C_i , where $i = T_x.SI$;
2. skips every operation O_y in $TM_{j[i]}$, where $T_y.TO \leq T_x.RTO$ (such O_y must have been executed at C_i before O_x 's generation, and hence causally before O_x ; the rest operations in $TM_{j[i]}$ must be concurrent with O_x);
3. divides the rest operations in $TM_{j[i]}$, without changing their orders, into two sublists: $L_1 + L_2$, where:
 - 1) L_1 contains every remote operation O_y , where $T_x.RTO < T_y.TO < T_x.TO$ (such O_y must be concurrent with and totally before O_x); and
 - 2) L_2 contains the rest of operations in $TM_{j[i]}$, which are local operations but have not yet been notified by the server; and they must be concurrent with and totally after O_x ;
4. transforms O_x against operations in L_1 symmetrically, i.e., $SLT(O_x, L_1) = (O_x\{L_1\}, L_1\{O_x\})$;
5. transforms $O_x\{L_1\}$ against operations in L_2 symmetrically, i.e., $SLT(O_x\{L_1\}, L_2) = (O_x\{L_1, L_2\}, L_2\{O_x\})$;
6. executes $O_x\{L_1, L_2\}$ at C_j ;
7. for every $TM_{j[k]}$, where $k \neq i$, maintained at C_j ,
 - 1) saves $O_x\{L_1\}$ at the position immediately after the last operation in $TM_{j[k]}$, whose $TO < T_x.TO$ (this position is at the border between operations that are totally before and after O_x in $TM_{j[k]}$);
 - 2) replaces the rest operations in $TM_{j[k]}$ with operations in $L_2\{O_x\}$ (those operations must be locally generated at C_j and totally after O_x).

As described in Steps 5 and 6, O_x is transformed only against operations in L_1 and L_2 , which are concurrent operations with O_x , as shown in the next section. This implies the time complexity of transforming O_x is linear to the number of concurrent operations.

6.2.5 CP2-Avoidance Validation

It can be shown that L_1 is an OSP1 with respect to O_x for $SLT(O_x, L_1)$, and L_2 is an OSP2 with respect to $O_x\{L_1\}$ for $SLT(O_x\{L_1\}, L_2)$ because:

1. All operations in L_1 and L_2 are:

- 1) totally ordered as they are put into transformation paths (in Step 3 of Local Processing and Step 8 (1) in Remote Processing) at the positions that are consistent with the total ordering determined by the server serialization; and their relative positions in transformation paths never change during local and remote processing;
 - 2) concurrent with O_x as they are operations not generated by C_i , according to the methods of putting operations in a transformation path; and they are not among those operations that have been executed at C_i when O_x was generated, according to the condition $T_y.TO \leq T_x.RTO$ for skipping O_y in Step 3 of Remote Processing;
 - 3) contextualized as they are put into the transformation paths in proper initial forms (original local operations in Step 3 of Local Processing, and properly transformed remote operations in Step 8 (1) of Remote Processing) and at right positions (determined by the total ordering), and their contexts are properly adjusted in symmetrical transformed (in *SLT*) with follow-up remote operations.
2. L_1 contains all operations that are concurrent and totally before O_x (as a transformation path contains all remote operations that are not generated by the client where O_x was generated), and $L_1[1] \stackrel{\cong}{=} O_x$ (according to Point 1).
3. L_2 contains operations that are concurrent and totally after O_x and causally related with each other (as operations in L_2 contains only local operations that have not yet be acknowledged for total ordering by the server), and $O_x\{L_1\} \stackrel{\cong}{=} L_2[1]$ (according to Point 1).

By Theorem 3, POT CP2-avoidance is validated.

6.2.6 Selective Replication and Garbage Collection

For conceptual simplicity, we have described POT under a basic *TM* management scheme, in which:

1. each local operation is replicated in $N - 1$ transformation paths (in Step 3 of Local Processing);
2. each remote operation is replicated in $N - 2$ transformation paths, i.e., all paths, except the one that is used for transforming the incoming operation (in Step 8-(1) of Remote Processing); and
3. all operations in $L_2\{O_x\}$ are replicated at other $N - 2$ transformation paths (in Step 8 (2) of Remote Processing).

In this scheme, the replication overhead (in both space and time) could be large when there is a large number N of collaborating users in a session. Fortunately, most replications could be avoided. Based on OSP1 and OSP2 properties established in Theorems 4 and 5 (Section 4.3), we have devised a *selective replication* scheme to avoid unnecessary replication, which is described below.

In addition to the $N - 1$ transformation paths in *TM*, we maintain a *History Buffer*, which contains a list of real copies (not references) of all operations that have been executed at this client. Operations saved in transformation paths will be

initially references (not copies) to the corresponding operations in *HB*; an actual copy is created in a transformation path only if the referenced operation in *HB* is a member of L_1 in *SLT* transformation with an incoming remote operation. Operations in *HB* are totally order and contextualized (i.e., a TOCOS by Definition 7); the current document state can be obtained by applying operations in *HB* sequentially to the initial document. The maintenance of the extended *TM* is sketched below:

1. After executing a local operation O_l generated at C_i ,
 - 1) a copy of O_l is appended at the end of *HB*;
 - 2) references to O_l in *HB* is saved at the end of each transformation path $TM_i[j]$, $1 \leq j \leq N$, $i \neq j$, at C_i (at Step 3 of Local Processing).
2. After dividing operations in a transformation path into L_1 and L_2 (at Step 4 (1) of Remote Processing), create a copy of each operation in L_1 from *HB* (unless a copy has been created for this operation before).
3. After executing a remote operation $O_r\{L_1, L_2\}$ (at Step 8 (1) of Remote Processing),
 - 1) a copy of $O_r\{L_1\}$ is inserted into *HB* at the position determined by its total ordering ($T_r.TO$);
 - 2) a reference to $O_r\{L_1\}$ in *HB* is inserted in every $TM_i[j]$, $1 \leq j \leq N$, $i \neq j$, $j \neq T_r.SI$, at the position determined by its total ordering ($T_r.TO$).

The rationale behind selectively replicating operations in L_1 is that, on the one hand, an *SLT* transformation (in Step 5 of Remote Processing) may change those operations in L_1 and hence corresponding operations in *HB*; on the other hand, according to Theorem 4, the same L_1 -referenced operation in *HB* may be needed for transforming multiple *concurrent* and *totally-after* operations generated from different sites, as L_1 is an OSP1 with respect to those operations. Therefore, it is necessary to create replicas in the corresponding transformation paths to avoid changing those operations in *HB*. However, there is no need for replicating operations in L_2 since no operation in L_2 will be transformed with two different operations under the same context according to Theorem 5. Moreover, replication of $L_2\{O_x\}$ in Step 8 (2) of Remote Processing can also be avoided as operation references in L_2 are adequate to represent L_2 in those transformation paths.

Operations in *TM* need not to be maintained for the whole session. For the purpose of consistency maintenance, an operation can be collected as garbage if no future operation will be concurrent with it, so it is no longer needed for future transformations [22]. In POT, garbage collection conditions are specified below:

1. An operation O_y (either copy or reference) in $TM_j[i]$ can be collected as garbage if $T_y.TO \leq T_x.RTO$, where T_x is the timestamp associated with O_x which was generated at C_i and has arrived at C_j . In other words, an operation in a transformation path can be collected as garbage if it is skipped by a remote operation in Step 3 of Remote Processing.
2. An operation in *HB* can be collected as garbage if its references from all transformation paths have been:
 - 1) removed as garbage, or
 - 2) replaced with replicas.

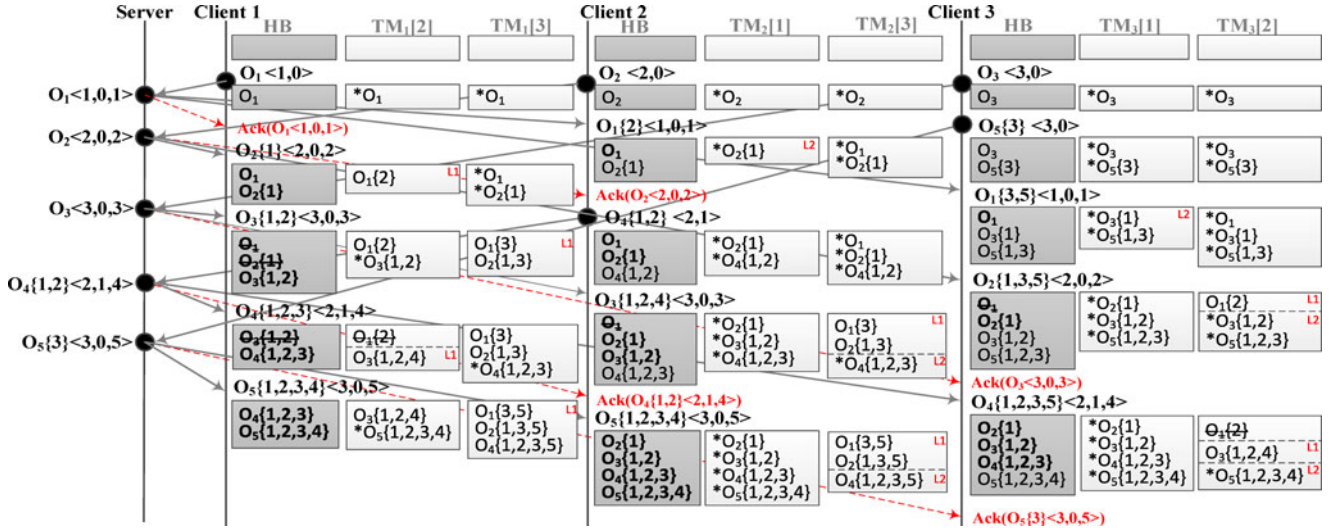


Fig. 6. POT: Each client maintains a TM consisting of $N - 1$ paths and a HB . Notations: $*O_x$ in a path represents a reference to an operation in HB ; $\emptyset$ in a path or HB represents an operation removed as garbage; $O_x < s, r, t >$ represents operation O_x with timestamp $< SI = s, RTO = r, TO = t >$. The contents in HB or a path represent the state *after* executing the current operation, as well as the state *before* executing the next operation; bold operations in HB represents those that have been totally ordered or acknowledged by the server. Operations propagated from remote sites are represented in their execution forms (after transformation).

6.2.7 An Example

To illustrate the working of the key schemes in POT, we use a scenario of five operations with causal relations: $((O_1 || O_2) \rightarrow O_4) || (O_3 \rightarrow O_5)$, and total ordering relations: $O_1 \Rightarrow O_2 \Rightarrow O_3 \Rightarrow O_4 \Rightarrow O_5$, in Fig. 6.

Consider operation $O_4\{1, 2\}$ at Client 2. Upon generation, it is executed immediately, timestamped with $< SI = 2, RTO = 1 >$, saved (a copy) at the end of HB and two references at the ends of $TM_2[1]$ and $TM_2[3]$, respectively, and propagated to the server. The server broadcasts $O_4\{1, 2\}$ with $< SI = 2, RTO = 1, TO = 4 >$ to all clients, including an $Ack(O_4\{1, 2\}, < SI = 2, RTO = 1, TO = 4 >)$ to Client 2.

When $O_4\{1, 2\}$ arrives at Client 3, $TM_3[2]$ is located; $O_1\{2\}$ is skipped and removed as garbage; the rest is split into: $L_1 = [*O_3\{1, 2\}]$ (reference), and $L_2 = [*O_5\{1, 2, 3\}]$ (reference). The reference in L_1 is replaced with an actual copy from HB ; $O_4\{1, 2\}$ is transformed with the copied operation in L_1 symmetrically to get $O_4\{1, 2, 3\}$ and $L_1\{O_4\} = [O_3\{1, 2, 4\}]$ (copy), but the corresponding copy in HB remains unchanged. The reference in L_2 is *not* replaced with the actual copy; $O_4\{1, 2, 3\}$ is transformed with the actual operation in HB via the reference $*O_5\{1, 2, 3\}$ to get $O_4\{1, 2, 3, 5\}$ and $*O_5\{1, 2, 3, 4\}$, which refers to the changed copy $O_5\{1, 2, 3, 4\}$ in HB . After that, $O_4\{1, 2, 3, 5\}$ is executed on the local document. Then, $O_4\{1, 2, 3\}$ (a copy) is saved in HB , at the position of its total order and $*O_4\{1, 2, 3\}$ (a reference) is placed in $TM_3[1]$ at the position of its total order.

It is instructive to observe the evolution of memory contents in TM_1 at Client 1: After executing a local operation O_1 , a copy of O_1 is saved in HB , and a reference $*O_1$ is placed at the end of $TM_1[2]$ and $TM_1[3]$, respectively. After executing a remote operation O_2 , the reference $*O_1$ in $TM_1[2]$ is replaced by a copy of O_1 and transformed into $O_1\{2\}$. After executing a remote operation O_3 , the reference $*O_1$ in $TM_1[3]$ is replaced by a copy of O_1 and transformed into $O_1\{3\}$. At this moment, the original copy of O_1 in HB is removed as it has been copied to all paths; two different copies, $O_1\{2\}$ and $O_1\{3\}$, are maintained in $TM_1[2]$ and

$TM_1[3]$, respectively, which illustrates a case for replicating operations in L_1 , and garbage collection in HB .

A summary of all transformation patterns in the POT example are shown in Table 1, in which an operation O_x may be transformed with operations that is made of OSP1 (highlighted with *single-overlines*), and/or OSP2 (highlighted with *double-overlines*). Transformations happen at remote sites only.

7 CONCLUSION

In this work, we have conducted a comprehensive and in-depth study on achieving convergence in OT systems. The main outcomes of this work include: (1) discovery of CP2-avoidance conditions and transformation patterns underlying a wide range of OT systems; (2) validation of CP2-avoidance correctness of seven major OT systems; and (3) design of two new OT systems: TIBOT 2.0 based on a distributed total ordering scheme and capable of avoiding CP2 without using undo/redo; and POT capable of avoiding CP2 without using a vector-based scheme, or requiring a transformation-based server, or imposing any restriction on operation

TABLE 1
Transformation Patterns in the POT Example

Op.	Local	Server	Remote
O_1	Prop.	Prop.	Trans. with $\overline{O_2}$ @ Client 2
	as-is	as-is	Trans. with $\overline{O_3}, \overline{O_5\{3\}}$ @ Client 3
O_2	Prop.	Prop.	Trans. with $\overline{O_1}$ @ Client 1
	as-is	as-is	Trans. with $\overline{O_1}, \overline{O_3\{1\}}, \overline{O_5\{1, 3\}}$ @ Client 3
O_3	Prop.	Prop.	Trans. with $\overline{O_1}, \overline{O_2\{1\}}$ @ Client 1
	as-is	as-is	Trans. with $\overline{O_1}, \overline{O_2\{1\}}, \overline{O_4\{1, 2\}}$ @ Client 2
$O_4\{1, 2\}$	Prop.	Prop.	Trans. with $\overline{O_3\{1, 2\}}$ @ Client 1
	as-is	as-is	Trans. with $\overline{O_3\{1, 2\}}, \overline{O_5\{1, 2, 3\}}$ @ Client 3
$O_5\{3\}$	Prop.	Prop.	Trans. with $\overline{O_1\{3\}}, \overline{O_2\{1, 3\}}, \overline{O_4\{1, 2, 3\}}$
	as-is	as-is	@ Client 1, Client 3

propagation, which is particularly suitable for supporting large scale collaborations with an arbitrary number of dynamic collaborating users. It is amazing to see that all OT systems known to be able to avoid CP2 meet the same CP2-avoidance condition and follow the same transformation patterns, despite their highly diverse system structures, propagation protocols, and transformation algorithms. These results contribute to the advancement of OT knowledge, technology, and OT-based collaborative applications.

In this work, we have demonstrated that certain CP2-avoidance conditions and patterns are sufficient to avoid CP2; but these conditions and patterns may not be necessary for avoiding CP2 [30]. Seeking for alternative valid and desirable CP2-avoidance conditions and patterns is an interesting direction in future research as it may inspire invention of novel OT systems for meeting special algorithmic or application needs in the future.

This work was motivated by our curiosity to find out whether, how and why some existing OT systems are able to achieve convergence by CP2-avoidance, which resulted in the discovery of new CP2-avoidance conditions and patterns underlying a range of existing OT systems, which in turn led to invention of new OT systems. While continuing with curiosity-driven OT research, we devote significant efforts to applying invented OT technologies to real-world applications, which provide exciting opportunities and another driving force to future OT technological innovation.

ACKNOWLEDGMENTS

This research is partially supported by an Academic Research Fund Tier 1 Grant from MOE Singapore. The authors wish to thank anonymous reviewers for their insightful and constructive comments and suggestions. C. Sun is the corresponding author.

REFERENCES

- Agustina, F. Liu, S. Xia, H. F. Shen, and C. Sun, "CoMaya: incorporating advance collaboration capabilities into 3D digital media design tools," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 2008, pp. 5–8.
- Agustina and C. Sun, "Dependency-conflict detection in real-time collaborative 3D design systems," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 2013, pp. 715–728.
- J. Begole, M. B. Rosson, and C. A. Shaffer, "Flexible collaboration transparency: Supporting worker independence in replicated application-sharing systems," *ACM Trans. Comput.-Human Interact.*, vol. 6, no. 2, pp. 95–132, 1999.
- C. A. Ellis and S. J. Gibbs, "Concurrency control in groupware systems," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1989, pp. 399–407.
- S. Greenberg and D. Marwood, "Real time groupware as distributed system: Concurrency control and its effect on the interface," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 1994, pp. 207–217.
- N. Gu, J. Yang, and Q. Zhang, "Consistency maintenance based on the mark & retrace technique in groupware systems," in *Proc. ACM Conf. Supporting Group Work*, 2005, pp. 264–273.
- L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978.
- R. Li and D. Li, "A new operational transformation framework for real-time group editors," *IEEE Trans. Parallel Distrib. Syst.*, vol. 18, no. 3, pp. 307–319, Mar. 2007.
- R. Li, D. Li, and C. Sun, "A time interval based consistency control algorithm for interactive groupware applications," in *Proc. IEEE Int. Conf. Parallel Distrib. Syst.*, 2004, pp. 429–436.
- M. MacFadden, "The client stop and wait operational transformation control algorithm," *SOLUTE Consulting*, San Diego, USA, 2013.
- D. Nichols, P. Curtis, M. Dixon, and J. Lamping, "High-latency, low-bandwidth windowing in the Jupiter collaboration system," in *Proc. 8th Annu. ACM Symp. User Interface Softw. Technol.*, 1995, pp. 111–120.
- A. Prakash and M. Knister, "A framework for undoing actions in collaborative systems," *ACM Trans. Comput.-Human Interact.*, vol. 1, no. 4, pp. 295–330, 1994.
- G. Oster, P. Molli, P. Urso, and A. Imine, "Tombstone transformation functions for ensuring consistency in collaborative editing systems," in *Proc. IEEE Conf. CollaborateCom*, 2006, pp. 1–10.
- M. Ressel and R. Gunzenhauser, "Reducing the problems of group undo," in *Proc. ACM Conf. Supporting Group Work*, 2005, pp. 131–139.
- M. Ressel, N. Ruhland, and R. Gunzenhauser, "An integrating, transformation-oriented approach to concurrency control and undo in group editors," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 1996, pp. 288–297.
- R. Schwarz and F. Mattern, "Detecting causal relationships in distributed computations: in search of the Holy Grail," *J. Distrib. Comput.*, vol. 7, no. 3, pp. 149–174, 1994.
- B. Shao, D. Li, and N. Gu, "A fast operational transformation algorithm for mobile and asynchronous collaboration," *IEEE Trans. Parallel Distrib. Syst.*, vol. 21, no. 12, pp. 1707–1720, Dec. 2010.
- H. F. Shen and C. Sun, "Flexible notification for collaborative systems," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 2002, pp. 77–86.
- D. Spiewak. (2010). Understanding and applying operational transformation. [Online]. Available: <http://www.codecommit.com/blog/java/understanding-and-applying-operational-transformation>
- C. Sun and P. Maheshwari, "A single phase protocol for total and causal ordering of group operations in distributed systems," *J. Comput. Inf.*, vol. 2, no. 1, pp. 219–237, 1996.
- C. Sun and C. A. Ellis, "Operational transformation in real-time group editors: issues, algorithms, and achievements," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 1998, pp. 59–68.
- C. Sun, X. Jia, Y. Zhang, Y. Yang, and D. Chen, "Achieving convergence, causality-preservation, and intention-preservation in real-time cooperative editing systems," *ACM Trans. Comput.-Human Interact.*, vol. 5, no. 1, pp. 63–108, 1998.
- C. Sun, S. Xia, D. Sun, D. Chen, H. Shen, and W. Cai, "Transparent adaptation of single-user applications for multi-user real-time collaboration," *ACM Trans. Comput.-Human Interact.*, vol. 13, no. 4, pp. 531–582, 2006.
- C. Sun. (2011). OTFAQ: Operational transformation frequent asked questions. [Online]. Available: <http://cooffice.ntu.edu.sg/otfaq>
- D. Sun and C. Sun, "Context-based operational transformation in distributed collaborative editing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 20, no. 10, pp. 1454–1470, Oct. 2009.
- C. Sun, H. Wen, and H. Fan, "Operational transformation for orthogonal conflict resolution in collaborative two-dimensional document editing systems," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 2012, pp. 1391–1400.
- C. Sun, Y. Xu, and Agustina, "Exhaustive search of puzzles in operational transformation," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 2014, pp. 519–529.
- N. Vidot, M. Cart, J. Ferrie, and M. Suleiman, "Copies convergence in a distributed real-time collaborative environment," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 2000, pp. 171–180.
- D. Wang, A. Mah, and S. Lassen. (2010). Google wave operational transformation. [Online]. Available: <http://www.waveprotocol.org/whitepapers/operational-transform>
- Y. Xu, C. Sun, and M. Li, "Achieving convergence in operational transformation: conditions, mechanisms and systems," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 2014, pp. 505–518.



Yi Xu is working towards the PhD degree at the School of Computer Engineering, Nanyang Technological University, Singapore. Her research interests include computer-supported cooperative work, distributed systems, and system verification and validation.



Chengzheng Sun received the PhD degree in computer engineering from the National University of Defense Technology, China, 1987, and the PhD degree in computer science from the University of Amsterdam, The Netherlands, 1992. He is a professor in the School of Computer Engineering, Nanyang Technological University, Singapore. His current research lies in the intersections of computer-supported cooperative work and distributed systems.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.